Where Does Your Data Live?

Lecture Notes — Week 1: Structure, Pointers, Dynamic Memory, and Simple Contracts

Auqib Hamid Lone

Department of Computer Science & Engineering
Government College of Engineering & Technology (GCET), Ganderbal

August 13, 2026

1.1 Why Data Organization Is the Subject

Read alongside: Horowitz & Sahni, *Fundamentals of Data Structures*, Ch. 1 (chapter-level); Aho, Hopcroft & Ullman, *Data Structures and Algorithms*, Ch. 1 (chapter-level) [1].

Before anything else, an honest inventory of what you already have. You know that variables store values, that an array stores several values in a row, that a function divides a program into pieces, that a loop repeats work, and that C gives you `struct` for grouping fields. This course does not re-teach any of that. What it adds is a single question, asked in a new way every week: *what arrangement of data fits this job?* Four narrower versions of that question organize this very chapter: how do we organize a program's *logic* before we organize its data; where does a running C program keep its data; how can we use memory that we request while the program runs; and how can we describe a data structure before implementing it. Compressed to a plan: structure the logic, draw the memory, use one pointer, write one simple contract — in that order, which is also the order of the sections below.

Consider a concrete instance of why the question matters. A loop that scans student records one by one always finds the right student, and with 5 records it is instant. With 10 million records, that same correct loop is far too slow. The answer stays right; the *arrangement* of the data is the difference. This course is the catalogue of such arrangements and, more importantly, of the judgement to choose between them [2, 1]. **Data structures keep a correct program useful as the data grows.**

The course runs twelve weeks, and every week repeats one shape: *state the contract, then implement it, then analyse what it costs.* Weeks 1–2 are Foundations: structure, memory, pointers, and cost. Weeks 3–5 take up the linear structures: arrays, stacks, and queues. Weeks 6–7 turn to linked structures: singly and doubly linked lists. Weeks 8–9 cover finding and ordering: searching and sorting. Week 10 is files and hashing: file organization and hash tables. Weeks 11–12 close with connected data: trees, AVL trees, and graphs.

One running thread carries through the whole term rather than twelve unrelated topics. The running example is a small expression processor that reads `(a+b)*c`: check its brackets, convert it, evaluate it, and look up the identifiers it uses. A **stack** checks the brackets (Weeks 3–4). A **symbol table** stores the identifiers (Weeks 6, 10). An **expression tree** gives the evaluation (Week 11). Not twelve unrelated topics — one program learning new tricks each week.

A program, in the shape this course cares about, is algorithms operating on data structures. Structured programming — which you have already met in some form, and which Section 1.2 makes precise — disciplines the first term. This course disciplines the second.

1.2 Structured Programming: Disciplining the Logic Before the Data

Read alongside: standard treatment; see Horowitz & Sahni, Ch. 1 (chapter-level) [2].
Definition (Structured programming). *The discipline of building a program from exactly three control patterns — sequence, selection, and iteration — instead of unrestricted jumps.*

The three patterns, stated plainly: **sequence** means doing one thing, then the next; **selection** means choosing exactly one branch, written in C as `if/else`; **iteration** means repeating work while a condition holds, written as `for` or `while`. Every C program you have written is built from nothing but these three shapes, however elaborate it looks.

Example 1.2.1 (One task, three patterns)**.** *Consider the task of printing the larger of two exam marks.*

```
int larger(int a, int b) {     // sequence: read a, b
  if (a > b)                   // selection
    return a;
  return b;
}
```

Reading the function against the three patterns: the parameter list `(int a, int b)` *is filled in by* sequence *— the two values simply arrive, one after the other, before anything else happens. The* `if (a > b)` *test is* selection*: exactly one of the two* `return` *statements executes. No loop appears anywhere in this function — comparing two values needs only sequence and selection. A loop appears the moment the task changes to comparing* more than two *marks, because then the same comparison must repeat once per additional value.*

1.2.1 Why structure the program before structuring the data

The reason this course opens with a program-organization idea rather than a data-organization idea is that the same discipline is about to be reapplied to data, one level up. A function is a **named, reusable block** — exactly the idea this course will apply to data: name a contract, then implement it. Top-down design proceeds the same way for programs as it will for data structures: state the goal, break it into smaller pieces, then fill each piece in. And there is a practical payoff specific to this course's subject matter: well-structured code is what makes pointer and memory bugs *findable* — a tangled program hides them, because a bug in an unstructured jumble of jumps could be almost anywhere, while a bug in a disciplined sequence of named functions is localized to one function's sequence/selection/iteration logic. This course reuses that same discipline for data: state the contract (the ADT) before choosing the implementation.

Before turning to any actual structure, though, the more basic question remains unanswered: where does a program even *put* its data? That is the subject of the next section.

1.3 The Memory of a Running Program

Read alongside: Wirth, *Algorithms and Data Structures*, §4.2, p. 111 — the pointer/dynamic-allocation argument; Horowitz & Sahni, Ch. 1 (chapter-level).

Start from the smallest possible example. The declaration `int marks = 80;` stores the value `80` somewhere in memory, and that variable has three things attached to it: a **name** (`marks`), a **value** (`80`), and an **address** — its actual place in memory. An address is like a house number: it

tells us where to look. Every variable in every program you will ever write carries this same triad, whether or not you ever think about the third piece explicitly.

The honest answer to *where*, for this whole course, is a region of memory with a simple two-part geography that we never abandon. Figure 1.1 draws it the way every later diagram in the course will: the **stack** above for short-lived local variables, the **heap** below for memory requested at run time.

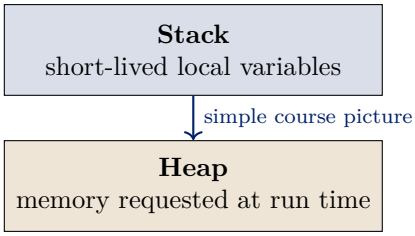

Figure 1.1: The two places a running C program keeps data: the stack for short-lived local variables, the heap for memory requested at run time. The whole course reuses this orientation.

1.3.1 The stack: automatic storage

A variable declared inside a function is created when the function starts, is available while that function is running, and disappears when the function returns. The program manages this basic lifetime automatically — no explicit allocation, no explicit cleanup. Writing `int score = 80;` inside a function is a local variable with exactly this lifetime.

To see what "automatic" really means, trace what happens to the stack as one function calls another. The deck does this with a small pair of functions: `main` holds `int a = 5;`, and calls `add(x, y)` that computes `int s = x + y;` and returns it.

Example 1.3.1 (Stack growth, Step 1: `main` starts)**.** *When the program starts, the system creates one* ***frame*** *on the stack for* ***`main`****, holding that function's local variables — here just* ***`int a = 5;`****. At this instant only this one frame exists (Figure 1.2). The space below it is free, waiting for the next function's frame.*

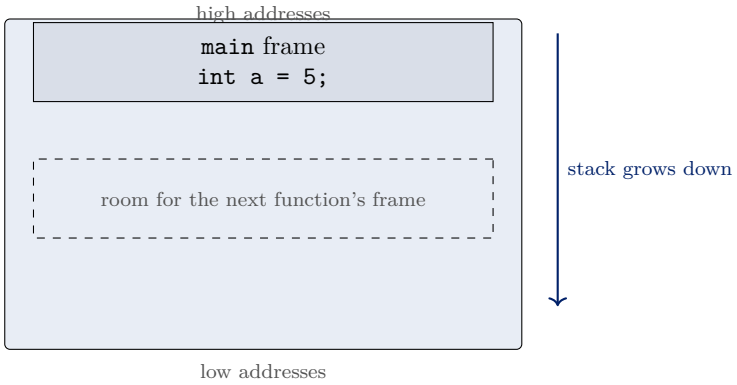

Figure 1.2: Stack growth, Step 1: `main` starts and its frame is created, holding `int a = 5;`. The stack grows *downward* — toward lower addresses.

Example 1.3.2 (Stack growth, Step 2: `add` is called)**.** *The call* ***`add(x=5, y=3)`*** *pushes a* second *frame just below* ***`main`****'s, holding the parameters (****`x`****,* ***`y`****) and the local* ***`int s = x + y;`*** *— which evaluates to* ***`8`*** *(Figure 1.3). Every new function call pushes a new frame just below the current one, so each new frame sits at a* ***lower*** *address than the frame above it.*

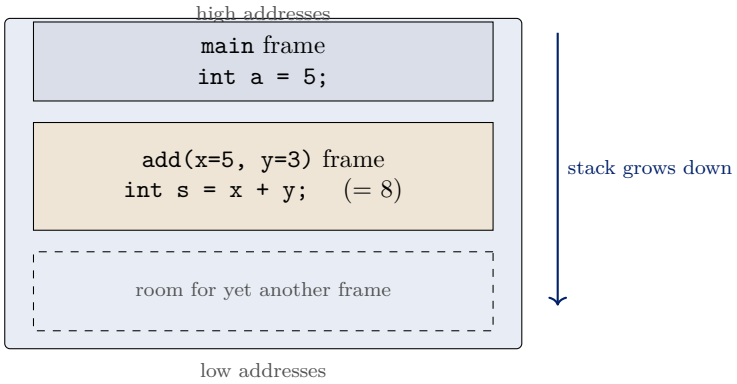

Figure 1.3: Stack growth, Step 2: the call to `add` pushes a second frame (parameters `x`,`y` and local `s`) at a lower address than `main`'s frame.

Example 1.3.3 (Stack growth, Step 3: `add` returns)**.** *When* ***`add`*** *returns, its frame is popped off the stack — it no longer exists, and its memory is available again for the next call (Figure 1.4). The answer* ***`8`*** *is copied back into* ***`main`****'s frame as* ***`int s = 8;`****. The stack has returned to its Step 1 shape.*

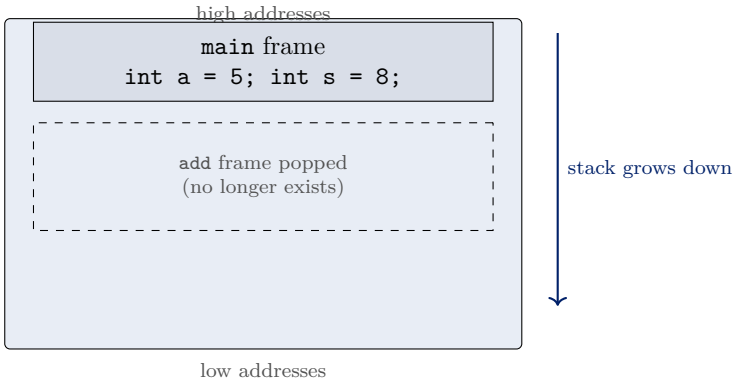

Figure 1.4: Stack growth, Step 3: `add` returns, its frame is popped and its memory is free again; `8` was copied back into `main`'s frame.

The three steps together are the whole story of automatic storage: lifetime is decided by the compiler, not by you. A local variable lives exactly as long as its enclosing call. This is precisely why returning a pointer to a local variable is a bug — the frame it pointed into ceases to exist the instant the function returns.

1.3.2 Why the stack is not enough

Consider a requirement the stack simply cannot meet: the user enters the number of records only after the program starts, and we may need space for 3 records today and 300 tomorrow. The stack

decides every frame's size at compile time and discards every local when its call returns. So the stack fails on both counts: we do not know the size in advance, and we may want the records to remain after the helper function that created them returns.

What we need is storage whose size is decided at run time and whose lifetime *we* control rather than the compiler. That is exactly what the **heap** provides, and the only way to reach the heap is through a pointer. Wirth puts the underlying argument crisply: recursive (variable-sized) structures "cannot be assigned a fixed amount of storage," so the compiler cannot associate specific addresses with them; the resolution is *dynamic allocation* — allocating store to components when they come into existence during program execution — with fixed storage reserved only for the *address* of the component [5] (p. 111).

Example 1.3.4 (A prediction worth making before the vocabulary)**.** *Two declarations:*

- `int score = 80;`
- `int *p = malloc(sizeof(int));`

Which variable can outlive the function that created it? The local variable `score` *belongs to its function call and dies with it. The heap block — the one requested by* `malloc` *— remains until the program releases it with* `free`*. The pointer* `p` *is only the* handle*; the block is the data storage. Answer: the* block*, not the variable that names it. The exact call that makes this happen (*`malloc` *and* `sizeof`*) comes in Section 1.5.*

1.4 Pointers: The Handle on Memory

Read alongside: Wirth, §4.2, p.111 — the dynamic-allocation argument and the address-as-handle picture.

Definition (Pointer). *A variable that stores an address. Its type records what kind of object resides at that address.*

An ordinary variable stores a value such as `80`. A pointer stores a *place* such as "where the score lives." The type tells C what kind of value is at that place. The declaration `int *p;` means: `p` can point to an `int` [5].

Figure 1.5 draws the distinction. The variable `p` on the left does not contain the score itself — it contains an address. The box on the right holds the actual value `80`. The arrow labelled "points to" is the whole idea: `p` tells us where the score can be found.

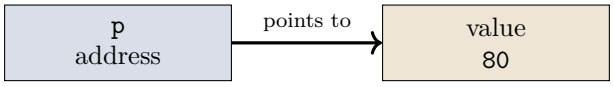

Figure 1.5: A pointer picture: `p` holds an address; the value `80` lives at that address. The pointer is the handle, not the data.

1.4.1 Pointers reach structures

Pointers earn their keep when they point at *structures*. The basic building block for most of this course is a two-field node:

```
struct node {
  int data;
```

```
  struct node *next;
};
```

The `data` field stores the value; the `next` field stores the address of *another* node. If there is no next node, set `next = NULL`. This is the seed of a linked list, studied properly in Week 6; here it appears only so that we can point at something real. The list handle will be called `head` starting Week 6; here the node is just `p`. For the field-access we prefer the arrow form: writing `p->data` reads the `data` field of the node `p` points at, and `p->next` reads its successor link. (The equivalent `(*p).data` is correct C but noisier; this course uses `->` throughout.)

1.4.2 The safe empty pointer

Definition (`NULL`). *The address value meaning "points at nothing." A pointer set to `NULL` is currently empty; reading through it is undefined behaviour — in practice, a crash.*

`NULL` is the agreed way to show that a pointer is not pointing to a valid object. Use it to mark empty state, check a pointer before using the object it should point to, and never read through an empty pointer. The canonical example of its meaning: `struct node *p = NULL;` says "this pointer reaches nothing yet." One notation convention holds for the whole term: the null pointer is always written `NULL` in uppercase, and the list handle is always named `head` starting Week 6.

1.5 One Heap Block

Read alongside: Wirth, §4.2, p.111 (dynamic allocation); Horowitz & Sahni, Ch. 1 (chapter-level).

This section does one thing repeatedly: allocate a block on the heap, use it, and return it. The whole memory-discipline of the course is these three moves, and the section works through C's whole heap-allocation interface — `malloc`, `sizeof`, `calloc`, `realloc`, and `free` — in the order a program actually needs them.

1.5.1 Requesting memory: `malloc` and `sizeof`

The call `malloc(n)` asks the heap for `n` *bytes* of memory and returns their address, or `NULL` if the request cannot be met. You should never count those bytes by hand; the `sizeof` operator computes them for you and stays correct when the type or the target machine changes. `malloc` itself never knows how big, say, an `int` is on its own — `sizeof` is what tells it, and every allocation request in this course uses it: `sizeof(int)` is typically `4`, and `sizeof(struct node)` is the size of the `data` field plus the `next` field together.

Example 1.5.1 (The basic pattern: one integer on the heap)**.**
```
int *p = malloc(sizeof(int));
if (p == NULL) return 1;
*p = 80;
```

Reading it line by line:

1. *`malloc(sizeof(int))` requests from the heap exactly as many bytes as one `int` needs, and returns their address.*
2. *`p` stores that address. If the request failed, `malloc` returns `NULL`; the `if` guard converts that failure into a controlled exit instead of a crash later.*

3. *`*p = 80;` writes the value `80` into the heap block — the block* at *the address `p` holds. (Note the difference from the pointer picture in Figure 1.5: there `p` pointed at a pre-existing box; here the box did not exist until this call.)*

The name `p` is not the heap block; the block is at the address `p` contains.

1.5.2 Zero-initialized memory: `calloc`

Definition (`calloc`). *`calloc(n, size)` allocates space for `n` items of `size` bytes each, and sets* every byte *of that space to `0`.*

```
int *counts = calloc(5, sizeof(int));
if (counts == NULL) return 1;
```

The declaration above requests room for five integers, exactly as `malloc(5 * sizeof(int))` would, but with one added guarantee: **`calloc` zero-initializes; `malloc` does not.** That guarantee is the entire difference between the two calls — same style of request, same `NULL` check, different starting content.

Example 1.5.2 (`malloc` vs. `calloc`: what is actually in the block)**.** *Figure 1.6 draws two independent four-byte requests side by side. On the left, `malloc(4 * sizeof(int))` hands back four bytes whose content is **unspecified** — drawn as `?` because the memory could contain anything left over from a previous use of that space; reading those bytes* before writing them *is undefined behaviour. On the right, `calloc(4, sizeof(int))` hands back four bytes already **set to zero**; a program may read them immediately, before ever writing to them, and the value it gets back is well-defined: `0`. The two calls request the same amount of memory; only the starting content differs, and only `calloc` pays the (small) extra cost of writing zeros before handing the block back.*

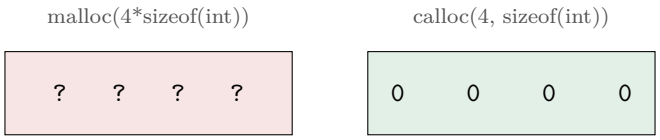

Figure 1.6: `malloc` vs. `calloc`: `malloc` hands back four unspecified bytes (reading before writing is undefined behaviour); `calloc` hands back four bytes already zeroed.

1.5.3 Resizing a block: `realloc`

Definition (`realloc`). *`realloc(p, newsize)` resizes the block `p` points to, preserving its existing content up to the smaller of the old and new sizes. The block **may move** — always keep the returned pointer.*

```
int *bigger = realloc(p, 10 * sizeof(int));
if (bigger == NULL) return 1;  // p is still valid here
p = bigger;
```

Two details in this idiom matter more than they look. First, the check is against `bigger`, not against `p` — if `realloc` fails, it returns `NULL` but leaves the *original* block at `p` untouched, so overwriting `p` with a possibly-`NULL` result before checking would silently lose access to still-valid memory. Second, `p` is only reassigned to `bigger` *after* the check succeeds.

Example 1.5.3 (Growing an array with `realloc`). *Start with `p = malloc(3 * sizeof(int))` holding `10 20 30`. Trace what happens, one statement at a time, when the array needs to grow to five slots:*

1. *`p = realloc(p, 5 * sizeof(int));` — the request asks for a block of five integers' worth of space, built from the existing block at `p`.*
2. *The first three ints (`10`, `20`, `30`) are* **copied** *into the new block, in the same order, by `realloc` itself — the caller does not write this copy.*
3. *The two new slots (index 3 and index 4) are* **unspecified** *— `realloc` does not zero new space; only `calloc` zeros space. A program that needs the new slots to start at zero must set them explicitly after the call.*

If `realloc` returns `NULL`, the original block at `p` is untouched and still valid — never overwrite `p` with the raw result before checking it, exactly as in the idiom above.

1.5.4 Allocate a complete node: the safer engineering pattern

Example 1.5.4 (Allocate a complete node: the safer engineering pattern).

```
struct node *p =
    malloc(sizeof(struct node));
if (p == NULL) return 1;
p->data = 42;
p->next = NULL;
```

This is the pattern the course will reuse every week:

1. *`sizeof(struct node)` asks for exactly one node's bytes — not a hand-counted `8` or `12`, which would be wrong the moment a field is added or the code is recompiled for another machine.*
2. *The result is checked against `NULL` before anything touches the block.*
3. *`p->data = 42;` stores a value in the node's `data` field.*
4. *`p->next = NULL;` makes the node's* end *explicit. The node has a clear stop: the next link is empty.*

Initialising every field makes the node's state predictable; an uninitialised field would contain garbage.

1.5.5 Audit the node before you ship it

A small program is not finished when it compiles. The deck supplies a four-point review checklist that doubles as this course's allocation invariant, and it is worth internalising as a habit:

1. Was the allocation checked against `NULL`?
2. Was every field initialised before the node was used?
3. Is there exactly one clear owner responsible for `free(p)`?
4. Can the program reach the node later, or has its pointer been lost?

The engineering habit behind the list: test both the normal path and the allocation-failure path. In C, out-of-memory is not an exception the runtime raises for you; it is a `NULL` return you must notice.

1.5.6 Trace it before running it

Reading C and running C are different skills. Before running the two lines

```
int *p = malloc(sizeof(int));
*p = 80;
```

the deck asks: *after these lines, what is true?* Answer, state by state:

1. `p` contains an address — the address `malloc` returned.
2. The heap block at that address contains `80` — written there by `*p = 80;`.
3. The name `p` itself is *not* the heap block. They are two different objects in two different regions (Figure 1.7).

The challenge in the same frame: *which line would fail if `p == NULL`?* Line two. Dereferencing `NULL` with `*p` is undefined behaviour — in practice a crash. Line one would have already told you the allocation failed, if you had checked.

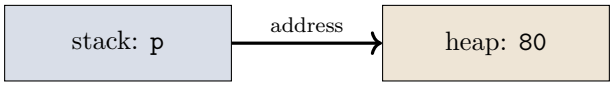

Figure 1.7: What happened after the two lines: `p` lives on the stack and holds the address of a heap block containing `80`. The pointer and the block are separate objects in separate regions.

1.5.7 How the heap grows

Just as the stack picture was built from three steps, the heap gets the same treatment. The growing arrow runs *upward*: the heap grows toward higher addresses, opposite to the stack.

Example 1.5.5 (Heap growth, Step 1: one `malloc` for three integers)**.** *One `malloc` request with room for three integers grew the used heap upward, placing a block that holds `10, 20, 30` (Figure 1.8). The block survives after the allocating function returns — that is the heap's whole point.*

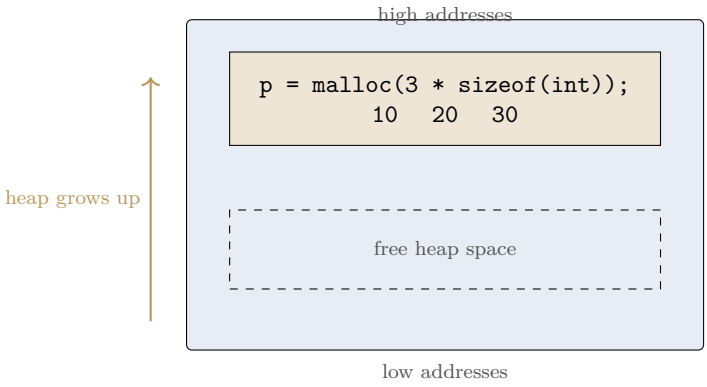

Figure 1.8: Heap growth, Step 1: `malloc(3 * sizeof(int))` places one block holding `10`, `20`, `30`; the used heap grew upward.

Example 1.5.6 (Heap growth, Step 2: a second block)**.** *The second request places* `q`*'s block at a* ***higher*** *address than* `p`*'s (Figure 1.9). Blocks are independent: each keeps its own data, and one block's values cannot reach into the other's.*

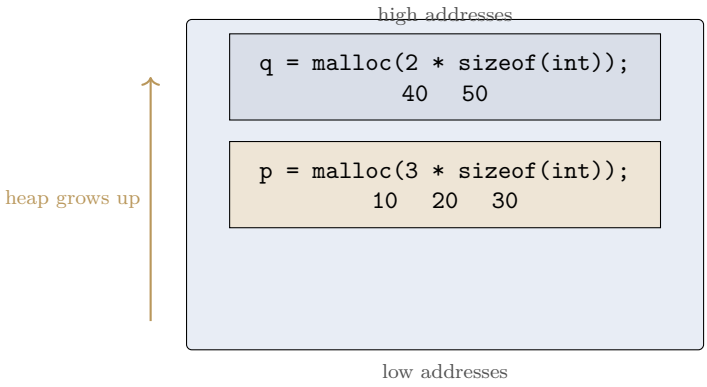

Figure 1.9: Heap growth, Step 2: a second `malloc` places `q`'s block at a higher address; the two blocks are independent.

Example 1.5.7 (Heap growth, Step 3: `free` returns a block)**.**
```
free(p);
p = NULL;
```

`free(p)` gives `p`'s block back to the system — it is no longer usable — and the used heap can shrink again (Figure 1.10). `q`'s block is untouched. Setting `p` to `NULL` afterwards makes the old pointer visibly empty, so it can never be mistaken for a live handle.

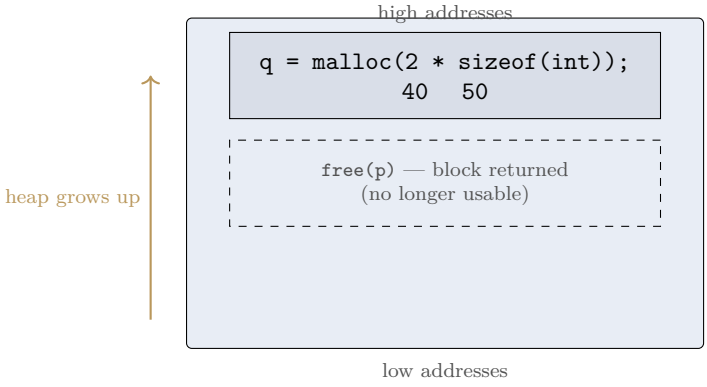

Figure 1.10: Heap growth, Step 3: `free(p)` returns `p`'s block to the system; `q`'s block is untouched.

1.5.8 Returning the memory

`free` is the counterpart of `malloc`, `calloc`, and `realloc` alike: every block obtained from any of the three needs a matching `free`. The recommended sequence is two lines that travel together:

```
free(p);
p = NULL;
```

The first line returns the heap block to the system. The second makes the old pointer visibly empty, so the handle cannot accidentally be used as though the block still existed. Request memory, use it, return it. This three-word rule is the entire memory-discipline of the course, and the two failures that violate it are the two beginner mistakes.

1.5.9 Two beginner mistakes

- Memory leak: forget to call `free`; the block stays reserved for the whole program, and if the pointer is overwritten without freeing, nothing can ever reach the block again. Nothing crashes — the program simply grows until it dies.
- Dangling pointer: use a pointer after its block was freed. The block is gone (or reused), so reads return garbage and writes corrupt whatever now occupies that space.

The simple habit that prevents both: **keep track of who requested the memory and who will return it**. Decide the single owner at the moment of allocation, and that owner is responsible for exactly one `free`.

1.6 Simple Contracts

Read alongside: Sedgewick & Wayne, *Algorithms*, §1.2, p.64 — the data-type/ADT definition; §1.3, p.120 — bag as a collection type; Karumanchi, *Data Structures and Algorithms Made Easy*, §1.4, p.18 (PDF) — the two-part ADT definition and the commonly-used-ADTs list; Aho, Hopcroft & Ullman, Ch. 1 (chapter-level).

We can now build a structure and reach it through a pointer. The remaining difficulty is describing one without drowning in pointers — which brings us to the course's central abstraction.

Definition (Abstract data type (ADT)). *A description of what we can do with data, without saying how it is stored: a promise about operations.*

Sedgewick & Wayne put it as cleanly as anyone: a data type is "a set of values and a set of operations on those values" [4] (p. 64), and the *abstract* data type is that contract stated before any implementation decides how it is met. An ADT is a promise about operations; the implementation is the code and memory arrangement behind the promise; and different implementations can follow the same promise.

Karumanchi's treatment makes the same contract concrete by splitting it into two named parts: an ADT "consists of two parts: 1. Declaration of data[;] 2. Declaration of operations" [3] (p. 18, PDF). The two textbook framings agree: Sedgewick's "set of values and set of operations" is exactly Karumanchi's "declaration of data" and "declaration of operations."

Example 1.6.1 (A very small bag)**.** *The deck's running example is a* ***bag of integers*** *— one of Sedgewick & Wayne's three collection types (p. 120) [4] — supporting exactly three operations:*

1. *`insert(x)` — put `x` into the bag.*
2. *`member(x)` — answer whether `x` is present.*
3. *`size()` — report how many items are present.*

The contract does not yet say whether the bag uses an array or a list. That is the point: the contract deliberately withholds the implementation.

1.6.1 One contract, two possible implementations

Both a dynamic array and a chain of linked nodes can honour the bag contract. Figure 1.11 draws the relationship: the contract box on top, two implementation boxes below, connected by plain arrows.

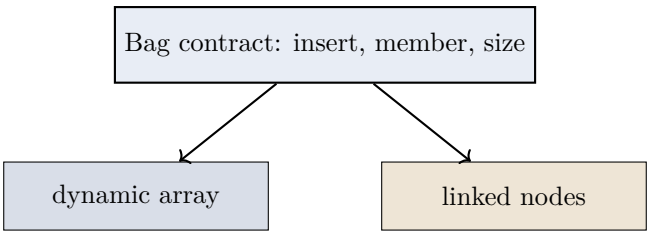

Figure 1.11: One contract, two implementations. Same operations, different internal arrangement. The caller's code does not change when the bottom box does — that substitutability is the value of the abstraction.

The key distinction, stated once and reused all term: **same operations, different internal arrangement**. This framing runs the whole course. Weeks 3 and 7 both implement a **stack**, once over an array and once over a chain of nodes. Weeks 5 and 7 do the same for a **queue**. Week 6 and Week 10 both implement a **lookup table** for the identifiers in our expression processor, first as a list and then as a hash table. Every time, the code that *uses* the structure is written against the contract and does not care which implementation it received.

1.6.2 This course is a tour of ADTs

The bag is a warm-up, not a one-off example: Linked Lists, Stacks, Queues, Priority Queues, Binary Trees, Hash Tables, and Graphs are all commonly used ADTs [3] (p. 18, PDF), and Weeks 3 through 12 work through this list one ADT at a time. Each week repeats the same three-beat shape — *state the question* ("What is a Stack?"), *state the contract* ("Stack ADT"), *then implement it* ("Implementation") — in that order, every time. Watch for this shape across the term: the contract is always stated and argued for before a single line of implementation is written.

Example 1.6.2 (Classifying contract statements vs. implementation statements)**.** *Is each statement part of the contract, or part of one implementation?*

1. *"`member(7)` answers yes or no." — contract. It describes what a user can ask.*
2. *"The values are stored in an array." — implementation. It describes* how *one particular bag is built.*
3. *"`size()` reports the number of values." — contract. It describes what a user can ask.*

The first and third describe what users can ask; the second describes how. This two-way split — what vs. how — is the habit to build now: ***state the contract before writing a line of the implementation****.*

1.7 What We Cannot Yet Answer

We can now build two implementations of the same ADT, and we have no vocabulary whatever for saying which is better — only "it felt faster," which is not an argument.

Week 2 supplies that vocabulary. **Algorithm analysis** counts an algorithm's basic operations as a function of the input size n, giving an exact cost function $T(n)$, then compresses that count into an asymptotic class — O, Ω, Θ — and separates the *best*, *worst*, and *average* case, so that two implementations can be compared without ever running either of them. The bag's two implementations differ *only* in cost; Week 2 is how we say that precisely.

Exercises

1. Rewrite `larger` from Section 1.2 so that it prints the largest of *three* exam marks instead of two, and identify which of the three control patterns (sequence, selection, iteration) your new version needs that the original did not.

2. If `p` lives on the stack and the node lives on the heap, what exactly is lost when `main` returns without calling `free`? Who could still reach that node?

3. Draw the complete picture for these four lines, then state what is true after each one:

```
struct node *p = malloc(sizeof(struct node));
if (p == NULL) return 1;
p->data = 7;
p->next = NULL;
```

   Your drawing should show which region each object lives in and the value of every field.

4. In the heap-growth trace, the second block (`q`) sits at a *higher* address than the first (`p`). If you now call `free(q)` and then `malloc` a block of three integers, what can you say about where the new block lands? (Hint: think about what "free space" the heap reuses, not about exact addresses.)

5. Suppose `p = calloc(3, sizeof(int));` and later `q = realloc(p, 6 * sizeof(int));` succeeds. After these two lines, what values are in slots 0–2 of the block `q` points to, and what can you say about slots 3–5? Justify each answer from the definitions of `calloc` and `realloc` given in Section 1.5.

Solutions

1. **Three marks, one new pattern:** a three-argument version, e.g.

```
int largest3(int a, int b, int c) {
  int m = a;                 // sequence
  if (b > m) m = b;          // selection
  if (c > m) m = c;          // selection
  return m;
}
```

   still needs no loop, because the count of marks (three) is fixed and known at compile time — two `if` statements suffice, so no new pattern is required. *Iteration* only becomes necessary once the number of marks is not fixed in the source code, e.g. reading marks from an array of unknown length and comparing each one in turn; that version would need a `for` loop over the array, which is exactly the boundary the deck's original frame points at: "a loop appears the moment we compare more than two marks" in the general case of an arbitrary count.

2. **What is lost:** the *reach*. `main`'s frame (which held `p`, the only pointer to the node) is popped when `main` returns, so the node's address disappears. The node itself is still allocated on the heap — it has not been freed — but no live pointer names it any more. Nobody can reach that node; it is a memory leak. The block stays reserved for the rest of the program's life.

3. **The complete picture:**
   - After line 1: `p` (on the stack) holds an address returned by `malloc`; a one-node block now exists on the heap, with garbage in both fields because nothing has written to it yet.
   - After line 2 (assuming the check passed): the address is known to be valid; execution continues.
   - After line 3: the heap node's `data` field holds `7`.
   - After line 4: the heap node's `next` field holds `NULL`, marking the end of the chain. The stack holds `p`; the heap holds a node with `data = 7`, `next = NULL`.
4. **The new block's location:** freeing `q` returns its two-integer block to the heap's free space. A subsequent `malloc` of three integers will be served from *available* free space; the exact address is the allocator's choice and not something the program may rely on. What you *can* say is general: the allocator reuses freed space, so the heap's used region can shrink and later regrow — the important invariant is that each block is reached only through a live pointer, never through a guess about its neighbour.
5. **`calloc` then `realloc`:** `calloc(3, sizeof(int))` zero-initializes its whole block, so immediately after the first line slots 0–2 all hold `0`. `realloc(p, 6 * sizeof(int))` preserves the existing content up to the smaller of the old and new sizes — the old size was three ints, so slots 0–2 are copied unchanged into the new block and remain `0`. But `realloc` is not `calloc`: it does *not* zero the newly added space, so slots 3–5 are **unspecified** — the same "garbage until written" state `malloc` produces, not guaranteed zero. A program that needs slots 3–5 to start at zero must set them explicitly after the `realloc` call, exactly as the `realloc` worked example in Section 1.5 notes for its own two new slots.

Summary

- A program is **algorithms** plus **data structures**. **Structured programming** disciplines the first term: every program is built from exactly three control patterns — sequence, selection, iteration — decomposed top-down into named, reusable functions; that same discipline (name the contract, then implement it) is what this course applies to data.
- **Stack** storage is automatic and scope-bound: a frame is pushed on call and popped on return, and the stack grows *down*. **Heap** storage is run-time sized and lifetime-controlled by you: it grows *up*, and its lifetime is controlled by explicit allocation and release rather than by function scope.
- A **pointer** stores an address — a handle on a value or structure, never the object itself. `NULL` means "points to nothing," and dereferencing it is undefined behaviour.
- The allocation idiom: `malloc` with `sizeof` for unspecified content, `calloc` when the block must start zero-initialized, `realloc` to resize an existing block (the block may move — always keep the returned pointer, and never overwrite the old pointer before checking the result), and `free` to return the block once it is no longer needed. Every request is checked against `NULL` before the block is used.
- The two silent failures: memory leak (never freed) and dangling pointer (used after being freed). Every allocation gets exactly one owner.
- An **ADT** is a contract about operations; a **data structure** is one implementation of it. State the contract before writing the code, and the rest of the course compares implementations of shared contracts.
- The rest of this course is a tour of ADTs — Linked Lists, Stacks, Queues, Priority Queues, Binary Trees, Hash Tables, and Graphs — each one: contract first, implementation second.

References

[1] Alfred V. Aho, John E. Hopcroft, and Jeffrey D. Ullman. *Data Structures and Algorithms*. Addison-Wesley, 1st edition, 1983.

[2] Ellis Horowitz and Sartaj Sahni. *Fundamentals of Data Structures*. Galgotia Booksource, 2nd edition, 2008.

[3] Narasimha Karumanchi. *Data Structures And Algorithms Made Easy*. CareerMonk Publications, 2017. This calibre-generated edition carries no printed folio numbers; page numbers cited in CS301 material are PDF page indices. NOT a source for Week 1's C-specific pointer/malloc content — see this course's supporting-books index for the coverage check.

[4] Robert Sedgewick and Kevin Wayne. *Algorithms*. Addison-Wesley, 4th edition, 2011.

[5] Niklaus Wirth. *Algorithms and Data Structures*. Author (freely distributed), 2004. Oberon version, August 2004; revision of the 1985 Prentice-Hall edition. Page numbers cited in CS301 material refer to this Oberon PDF, not to the 1985 print edition.